

Face + Voice Integrity

Kịch bản bảo vệ dự án

15 phút trình bày

10–15 phút demo

10 phút Q&A;

Trịnh Đức Dương

Do Quang Hiep

To Thanh Hai

Ngo Anh Duc

Bản hướng dẫn tập nói và điều khiển demo, ngày 03/10/2026. Câu chuyện mở đầu là tình huống giả định. Các số liệu lịch sử luôn được ghi rõ nguồn và thời điểm.

Nội dung: lời thoại từng slide, runbook demo 13 phút, phương án 10/15 phút và ngân hàng 23 câu hỏi phản biện.

Kịch bản nói: Face + Voice Integrity

Bản bảo vệ DDM501, cập nhật 03/10/2026. Nội dung đồng bộ với bộ PowerPoint [DDM501_Defense_15p_Demo13p_QA10p.pptx](#) trong cùng thư mục.

Khung thời gian: trình bày 15 phút, demo mục tiêu 13 phút (cho phép 10–15 phút), Q&A 10 phút. Tổng buổi bảo vệ 35–40 phút, bản đầy đủ 38 phút. Có 18 slide trình bày, 8 slide dẫn demo, 1 slide Q&A và 6 slide phụ lục. Lời thoại cũng nằm trong Notes của PowerPoint.

Kịch bản thao tác toàn diện: DEMO_RUNBOOK.md. Câu hỏi phản biện và phân vai: QA_GUIDE.md.

Cách sử dụng

- Tập nói theo ý, không đọc nguyên slide. Các đoạn dưới là lời thoại có thể đọc trực tiếp.
- Phần **Thao tác** dành cho người điều khiển máy, không đọc thành tiếng.
- Mốc thời gian là ngân sách cho giải thích và thao tác, không phải cam kết scheduler chạy đúng số giây.
- Câu chuyện mở đầu là tình huống giả định. Không nói nhóm đã phát hiện một vụ gian lận thật.
- Số liệu evidence ngày 03/10 là lịch sử đã lưu. Nếu mở run mới, đọc kết quả của chính run đó.
- Phụ lục được đánh dấu hidden trong trình chiếu. Mở bằng chọn slide khi cần trả lời câu hỏi.

Phân vai đề xuất

Người nói	Slide	Nội dung
Trịnh Đức Dương	1–5, 14–15; điều phối Q&A	Câu chuyện, giải pháp và lifecycle
To Thanh Hai	6–9, 16	Tích hợp, model I/O, kiến trúc, lưu trữ, CI/CD; điều khiển demo
Do Quang Hiep	10–13; phụ lục khi cần	Drift, decision engine, template và kiểm chứng evaluation
Ngo Anh Duc	17–18, dẫn demo 19–26	Giới hạn, chuyển phần, bằng chứng và demo

Nếu chỉ một người trình bày, bỏ các câu mời chuyển lời và giữ nguyên mạch nội dung.

Chuẩn bị trước buổi trình bày

1. Mở PowerPoint và dùng Presenter View để xem Notes. Đóng cửa sổ chứa key hoặc thông tin riêng.
2. Kiểm tra Docker Desktop và Airflow scheduler hoạt động. Không nâng cấp dependencies hoặc rebuild sát giờ.
3. Đăng nhập platform ở portal :18501 trước khi chiếu màn hình. Không hiển thị hoặc đọc API key.
4. Mở sẵn Airflow DAG [biometric_simulation](#) tại :18081 và MLflow simulation tại :15031.
5. Dùng nút **3. Khôi phục baseline demo** nếu có run trước đó. Không dùng [docker compose down -v](#).

6. Lưu sẵn evidence JSON của một lượt promotion và rollback thành công về mặt kịch bản. Slides 23 và 25 có evidence lịch sử dự phòng.
7. Đo thử thời gian hai kịch bản trên chính máy trình chiếu. Giới hạn demo tối đa 15 phút; nếu cần, dùng evidence lịch sử cho kịch bản thứ hai như DEMO_RUNBOOK hướng dẫn.

Lời thoại theo slide

Slide 01: Ai thực sự đang làm bài?

Người nói: Trịnh Đức Dương **Mốc thời gian:** Trình bày 00:00–00:25

Lời thoại

Kính chào thầy cô và các bạn. Nhóm em trình bày dự án Face + Voice Integrity. Điểm xuất phát của dự án là một câu hỏi rất gần với các kỳ đánh giá trực tuyến: người đang làm bài có đúng là nhân viên đã đăng ký hay không? Trong phần trình bày, nhóm sẽ đi từ tình huống kinh doanh tới cách xác minh, sau đó demo vòng đời MLOps khi dữ liệu thay đổi và khi một bản cập nhật cần bị thu hồi.

Thao tác / chuyển lời

Mở slide ở chế độ trình chiếu. Chưa mở dashboard hoặc nói tên các công cụ.

Nguồn đối chiếu: README.md; PROJECT_REQUIREMENTS.md

Slide 02: Một tài khoản đúng, một người làm bài khác

Người nói: Trịnh Đức Dương **Mốc thời gian:** Trình bày 00:25–01:40

Lời thoại

Xin mời thầy cô hình dung một doanh nghiệp tổ chức đánh giá ngoại ngữ thường niên. Lúc 8 giờ 55, một nhân viên đăng nhập bằng đúng tài khoản của mình. Hệ thống xác nhận tài khoản hợp lệ. Đến phần nói, một người khác có năng lực tốt hơn có thể hỗ trợ hoặc thực hiện thay. Khi kết thúc, hệ thống vẫn nhận được một bài làm gắn với đúng tài khoản. Nếu kết quả ấy được dùng để lập kế hoạch đào tạo hoặc phân công công việc, sai lệch danh tính sẽ làm sai lệch quyết định phía sau. Đến lúc có nghi vấn, HR lại thiếu bằng chứng tại thời điểm cần kiểm tra. Đây là tình huống giả định mà nhóm dùng để đặt yêu cầu thiết kế, không phải một vụ gian lận đã được nhóm xác minh. Từ đó, nhóm đặt vấn đề: làm sao bổ sung bằng chứng danh tính vào phần mềm thi đang có?

Thao tác / chuyển lời

Chỉ theo ba mốc thời gian. Dừng ngắn sau câu 'hệ thống vẫn nhận được một bài làm gắn với đúng tài khoản'.

Nguồn đối chiếu: PROJECT_REPORT.md: Câu chuyện business; SAAS_INTEGRATION.md

Slide 03: Ba khó khăn khiến việc xác minh cần được bổ sung

Người nói: Trịnh Đức Dương **Mốc thời gian:** Trình bày 01:40–02:25

Lời thoại

Nhóm nhìn thấy ba khó khăn liên quan với nhau. Thứ nhất là độ tin cậy của kết quả khi danh tính người làm bài có thể thay đổi. Thứ hai là khối lượng rà soát: có video hay audio chưa chắc đã giúp HR nhanh chóng tìm đúng thời điểm cần xem. Thứ ba là nguy cơ kết luận nhầm. Một nhân viên làm bài trung thực vẫn có thể bị điểm tương đồng thấp vì camera tối, microphone khác hoặc mẫu ghi danh đã cũ. Vì vậy, mục tiêu không chỉ là phát hiện thêm nghi vấn. Hệ thống còn phải trả được lý do, giữ được bằng chứng phù hợp và có cơ chế xử lý khi chính mô hình không còn hoạt động như trước. Nhóm chưa đo được mức giảm gian lận hay số giờ tiết kiệm tại doanh nghiệp thật.

Thao tác / chuyển lời

Nhấn vào khó khăn thứ ba để chuẩn bị cho câu chuyện drift ở phần sau.

Nguồn đối chiếu: PROJECT_REQUIREMENTS.md; ARCHITECTURE.md

Slide 04: Hệ thống kiểm tra những dấu hiệu nào?

Người nói: Trịnh Đức Dương **Mốc thời gian:** Trình bày 02:25–03:10

Lời thoại

Nhóm giới hạn bài toán vào danh tính và tính toàn vẹn của capture. Đối với thi hộ hoặc thay người, hệ thống so khớp khuôn mặt và giọng nói với mẫu của nhân viên đã ghi danh. Các detector bổ sung tìm dấu hiệu ảnh giả, âm thanh tổng hợp, nhiều khuôn mặt hoặc thay đổi người nói. Một tín hiệu khác là tái sử dụng chính xác media trong cùng phiên. Tuy nhiên, nhóm không xem những detector này là lời bảo đảm phát hiện mọi hành vi gian lận. Chúng chưa giải quyết đầy đủ đọc tài liệu ngoài màn hình, nhờ người gợi ý hoặc mọi loại deepfake. Vì thế, kết quả suspicious phải đi kèm lý do và được đặt vào quy trình xem xét của khách hàng, không biến thành quyết định kỷ luật tự động.

Thao tác / chuyển lời

Không gọi hệ thống là phần mềm chống gian lận toàn diện.

Nguồn đối chiếu: README.md; ARCHITECTURE.md; RESPONSIBLE_AI.md

Slide 05: Bổ sung xác minh vào phần mềm thi đang có

Người nói: Trịnh Đức Dương **Mốc thời gian:** Trình bày 03:10–03:50

Lời thoại

Doanh nghiệp thường đã có phần mềm thi và không muốn thay toàn bộ hệ thống. Vì vậy, nhóm chọn cách cung cấp một dịch vụ tích hợp. Khách hàng giữ bài thi, tài khoản, lịch capture, điểm và quyền quyết định. Dịch vụ của nhóm nhận ảnh và WAV, thực hiện kiểm tra rồi trả kết quả ngay qua API và gửi webhook. Portal giúp quản trị công ty xem lịch sử và bằng chứng. Nhịp gửi dữ liệu do khách hàng chọn, không phải hệ thống của nhóm liên tục stream toàn bộ phiên thi. Quy mô khoảng năm mươi nghìn nhân viên là mục tiêu để định hướng thiết kế dữ liệu và vận hành, chưa phải kết quả đo tải. Tiếp theo, Hải sẽ giải thích luồng tích hợp và cách hệ thống triển khai.

Thao tác / chuyển lời

Chuyển lời: 'Mời Hải trình bày hành trình một lượt kiểm tra'.

Nguồn đối chiếu: SAAS_INTEGRATION.md; README.md

Slide 06: Một lượt kiểm tra đi qua năm bước

Người nói: To Thanh Hai **Mốc thời gian:** Trình bày 03:50–04:35

Lời thoại

Một lượt kiểm tra bắt đầu từ mẫu ghi danh đã được nhân viên cung cấp với sự đồng ý. Backend của công ty gửi capture cùng định danh nhân viên, phiên và request. Dịch vụ thực hiện kiểm tra identity và các tín hiệu integrity, lưu kết quả rồi trả lại qua API. Webhook có chữ ký HMAC giúp khách hàng xác minh nguồn gửi, nhưng receiver vẫn phải chống xử lý trùng vì cơ chế retry có thể giao một thông báo nhiều lần. Khi cần xem xét, quản trị công ty có lịch sử, lý do và evidence theo đúng tenant. Idempotency là chi tiết quan trọng: gửi lại cùng request và cùng payload không được tạo thêm một quyết định khác.

Thao tác / chuyển lời

Đi từ trái sang phải. Có thể nhắc verified, suspicious và inconclusive bằng lời.

Nguồn đối chiếu: api/app/checks.py; api/app/webhooks.py; SAAS_INTEGRATION.md

Slide 07: Model biến ảnh và giọng nói thành quyết định

Người nói: To Thanh Hai **Mốc thời gian:** Trình bày 04:35–05:30

Lời thoại

Đầu vào của nhánh Face là ảnh. YuNet định vị khuôn mặt và SFace tạo embedding. Đầu vào của nhánh Voice là WAV, ECAPA tạo embedding giọng nói. Các vector được chuẩn hóa rồi so với tập template của đúng người được khai báo bằng cosine similarity. Hệ thống lấy mức tương đồng tốt nhất với tập template và áp dụng ngưỡng riêng cho từng modality. Vì đây là so khớp một người với mẫu của chính họ, nó là xác minh một-một, không phải tìm người giống nhất trong toàn bộ nhân viên. Trong dự án, model được huấn luyện lại là policy ngưỡng. Nhóm không tuyên bố fine-tune lại SFace hoặc ECAPA. Điều này cũng giải thích vì sao champion và challenger có thể dùng chung score nhưng trả quyết định khác nhau.

Thao tác / chuyển lời

Nhấn rõ chữ 'threshold policy' để tránh hiểu nhầm retrain encoder.

Nguồn đối chiếu: api/app/biometrics.py; pipeline/modality_training.py; docs/MODALITY_LIFECYCLE.md

Slide 08: Serving, vòng đời model và monitoring dùng chung dữ liệu có kiểm soát

Người nói: To Thanh Hai **Mốc thời gian:** Trình bày 05:30–06:30

Lời thoại

Kiến trúc có ba luồng chính. Serving xử lý request của khách hàng và ghi kết quả vào PostgreSQL, lưu media nghi vấn vào MinIO khi cần. Luồng MLOps do Airflow điều phối, lấy snapshot có phiên bản, tạo policy và lưu thông tin đánh giá trong MLflow. Luồng monitoring thu các chỉ số dịch vụ và model để hiển thị trong Grafana, đồng thời định tuyến cảnh báo. Registry quản lý phiên bản model, còn database lưu trạng thái triển khai và routing. Chúng có vai trò khác nhau. Hệ thống hiện chạy trên một máy bằng Docker Compose. Riêng simulation có vùng dữ liệu và MLflow riêng để có thể minh họa lỗi mà không đổi policy production. Host và scheduler vẫn dùng chung nên nhóm không gọi đây là cách ly tài nguyên tuyệt đối.

Thao tác / chuyển lời

Chỉ ra vòng feedback giữa PostgreSQL và Airflow, không đọc danh sách công cụ liên tục.

Nguồn đối chiếu: ARCHITECTURE.md; docker-compose.yml; docs/SIMULATION.md

Slide 09: Lưu đúng bằng chứng và giới hạn quyền truy cập

Người nói: To Thanh Hai **Mốc thời gian:** Trình bày 06:30–07:10

Lời thoại

Dữ liệu được lưu theo mục đích. PostgreSQL chứa embedding JSON, phiên bản template, các lượt kiểm tra và audit. MinIO chứa artifact cùng evidence media nghi vấn. Project hiện không dùng embedding database chuyên dụng như một vector search engine, vì đường serving là truy xuất template theo định danh đã biết. Các lượt đọc lịch sử và tải evidence được ràng buộc tenant, nên một công ty không thể đọc dữ liệu của công ty khác. Raw enrollment không được giữ mặc định. Query embeddings phục vụ monitoring cũng có cơ chế opt-in và thời hạn lưu. Đổi lại, nếu không giữ vector hoặc không có đủ nhãn tin cậy, hệ thống phải chấp nhận chưa đủ dữ liệu để kết luận drift. Sau đây Hiệp trình bày cách kiểm chứng điều đó.

Thao tác / chuyển lời

Chuyển lời: 'Mời Hiệp giải thích vì sao score giảm chưa đủ để retrain'.

Nguồn đối chiếu: ARCHITECTURE.md; docs/MODALITY_LIFECYCLE.md

Slide 10: Score giảm có thể đến từ nhiều nguyên nhân

Người nói: Do Quang Hiep **Mốc thời gian:** Trình bày 07:10–08:00

Lời thoại

Quay lại tình huống một nhân viên trung thực bị từ chối nhiều hơn. Nếu chỉ nhìn score giảm và lập tức retrain, chúng ta có thể sửa sai chỗ. Camera mới thiếu sáng hoặc mic kém cần xử lý capture. Nếu chỉ những người có mẫu ghi danh rất cũ gặp vấn đề, cập nhật template có thể phù hợp hơn. Chỉ khi bằng chứng cho thấy policy thực sự suy giảm trên dữ liệu được review và nhiều cohort thì retrain mới đáng cân nhắc. Vì vậy, project không để một kiểm định thống kê duy nhất điều khiển pipeline. Drift engine tổng hợp nhiều lớp tín hiệu để chọn hành động phù hợp, đồng thời giữ trạng thái chưa đủ bằng chứng khi dữ liệu còn thiếu.

Thao tác / chuyển lời

Dùng ví dụ 'đổi microphone' để giải thích input drift thay vì đọc định nghĩa thống kê.

Nguồn đối chiếu: pipeline/drift_decision.py; docs/MODALITY_LIFECYCLE.md

Slide 11: Năm lớp bằng chứng để đánh giá drift

Người nói: Do Quang Hiep **Mốc thời gian:** Trình bày 08:00–09:05

Lời thoại

Lớp đầu là chất lượng input, ví dụ độ sáng và blur của ảnh hoặc RMS và tỷ lệ im lặng của audio. Project dùng PSI để so sánh phân phối. Lớp thứ hai là embedding drift, dùng RBF MMD bình phương trên vector đa chiều, không kiểm từng chiều một cách độc lập. Lớp thứ ba là score drift, tách genuine và impostor khi có nhãn tin cậy để theo dõi hướng dịch chuyển và khoảng cách tới ngưỡng. Lớp thứ tư đo performance thật trên reviewed samples, bao gồm chấp nhận sai và từ chối sai. Cuối cùng là phân tích theo tuổi template. Mỗi modality có báo cáo riêng. Các feature như SNR hoặc góc quay đầu chưa được đo trong implementation thì nhóm không đưa vào như thể đã có. Công thức và ngưỡng chi tiết nằm ở phụ lục để phục vụ câu hỏi sâu.

Thao tác / chuyển lời

Giải thích FMR là chấp nhận sai người và FNMR là từ chối nhầm người thật.

Nguồn đối chiếu: pipeline/drift_decision.py; api/app/biometrics.py; pipeline/lifecycle_config.json

Slide 12: Decision engine quyết định hành động theo bằng chứng

Người nói: Do Quang Hiep **Mốc thời gian:** Trình bày 09:05–10:10

Lời thoại

Bảng này thể hiện quyết định vận hành. Chất lượng capture thay đổi thì điều tra input. Embedding hoặc score drift nhưng performance vẫn ổn thì theo dõi. Template cũ suy giảm riêng thì đi sang workflow template. Retrain yêu cầu embedding và score drift kéo dài, performance có nhãn review suy giảm trên nhiều nhóm tuổi, cùng điều kiện dữ liệu mới và cooldown. Mặc định cần ba cửa sổ mới đủ điều kiện, không phải refresh cùng một cửa sổ ba lần. Một điểm quan trọng là trạng thái insufficient data. Nó không có nghĩa hệ thống khỏe và cũng không có nghĩa đang drift. Nó nói rằng bằng chứng hiện tại chưa cho phép tự động hành động. Nguyên tắc này giúp hạn chế việc pipeline chạy đẹp nhưng dựa trên nhãn do chính model tự suy ra.

Thao tác / chuyển lời

Không mô tả các quy tắc là chứng minh quan hệ nhân quả tuyệt đối.

Nguồn đối chiếu: pipeline/drift_decision.py; api/app/lifecycle_service.py

Slide 13: Cập nhật template và retrain có hai đường kiểm soát

Người nói: Do Quang Hiep **Mốc thời gian:** Trình bày 10:10–10:55

Lời thoại

Với template update, nguy cơ lớn là template poisoning. Nếu cứ chấp nhận một lần xác minh thành công rồi thay mẫu bằng capture vừa gửi, một sai sót có thể bị khuếch đại. Vì vậy, workflow yêu cầu nhiều quan sát trusted, capture đủ chất lượng, margin lớn, nhất quán và không xung đột danh tính. Candidate template được đánh giá trên holdout riêng trước khi đổi con trở active, đồng thời giữ phiên bản trước để rollback. Với retrain, monitoring không tự train ngay trong request. Nó tạo intent để Airflow điều phối theo từng modality. Snapshot có phiên bản, CV tách theo identity và holdout không dùng để chọn ngưỡng. Hai đường này cùng nhằm tránh biến một thay đổi thiếu bằng chứng thành thay đổi production. Dương sẽ tiếp tục với quy trình challenger và canary.

Thao tác / chuyển lời

Chuyển lời sang Dương, nhấn rằng dữ liệu mới không đồng nghĩa được quyền đưa vào training.

Nguồn đối chiếu: api/app/template_lifecycle.py; pipeline/modality_training.py; airflow/dags/biometric_ml_pipeline.py

Slide 14: Model mới phải vượt qua cả offline và traffic gate

Người nói: Trịnh Đức Dương **Mốc thời gian:** Trình bày 10:55–12:00

Lời thoại

Sau training, policy mới chỉ là candidate. Offline gate so sánh candidate với champion trên đúng cùng một holdout. Nếu đạt, candidate trở thành challenger và được thử ở shadow. Trong shadow, request vẫn nhận quyết định của champion, nhưng hệ thống ghi lại quyết định của cả hai policy. Sau đó mới tới canary, nơi một phần traffic thực sự dùng threshold của challenger. Chỉ khi tất cả stage đạt điều kiện thì challenger mới trở thành champion. Nếu không đạt, phiên bản đó được giữ lại cùng lý do rejection để audit. Alias champion do đó có ý nghĩa phiên bản được chọn để phục vụ, không chỉ là tên môi trường. Với hai modality, Face có thể giữ champion cũ trong khi Voice đang rollout phiên bản khác.

Thao tác / chuyển lời

Chỉ nhánh thất bại trong sơ đồ. Làm rõ score dùng chung còn threshold và decision có thể khác.

Nguồn đối chiếu: api/app/model_lifecycle.py; api/app/lifecycle_api.py; docs/MODALITY_LIFECYCLE.md

Slide 15: Mỗi canary stage đều cần đủ mẫu, thời gian và nhãn

Người nói: Trịnh Đức Dương **Mốc thời gian:** Trình bày 12:00–12:45

Lời thoại

Canary không chỉ là một thanh phần trăm đẹp trên dashboard. Request được phân tuyến ổn định theo cohort. Mỗi stage phải có đủ quan sát, thời gian và nhãn tin cậy, rồi kiểm tra cả security lẫn chất lượng. FMR là điều kiện nhạy cảm: không cho phép tăng so với champion và còn phải nằm trong budget tuyệt đối. FNMR có mức hồi quy nhỏ được cấu hình. Ngoài ra còn có latency policy, disagreement và cohort chất lượng. Nếu nhãn chưa về đủ thì stage chờ, không tự thông qua vì chưa thấy lỗi. Nếu gate thất bại, challenger traffic về không. Nhãn đến muộn được đánh giá theo tick hàng giờ nên nhóm không gọi đó là phản ứng tức thời trong mọi tình huống.

Thao tác / chuyển lời

Phân biệt ngưỡng pilot này với demo 200 request/stage và 5 giây tối thiểu.

Nguồn đối chiếu: pipeline/lifecycle_config.json; api/app/model_lifecycle.py

Slide 16: CI/CD cập nhật ứng dụng, lifecycle cập nhật policy

Người nói: To Thanh Hai **Mốc thời gian:** Trình bày 12:45–13:40

Lời thoại

Ở đây có hai kiểu triển khai cần phân biệt. Code ứng dụng đi qua GitHub Actions: quality trên Ubuntu, preflight trên Windows, build images rồi runner Linux trong WSL gọi PowerShell để triển khai bằng Docker Desktop. Phần này là thay container bằng Compose trên một host. Nó chưa phải canary của nhiều replica ứng dụng. Còn policy Face và Voice được thay đổi qua trạng thái lifecycle trong PostgreSQL và alias MLflow, có shadow, canary và rollback như vừa trình bày. Prometheus và Grafana quan sát cả hai lớp. Hai test preflight Windows phải thực thi và không skip, vì quality job xanh trên Ubuntu một mình không đủ chứng minh chúng đã chạy.

Thao tác / chuyển lời

Chuyển sang Đức để trình bày bằng chứng và điều phối demo.

Nguồn đối chiếu: .github/workflows/ci.yml; DEPLOYMENT.md; OPERATIONS.md

Slide 17: Những gì cần kiểm chứng trước khi triển khai cho doanh nghiệp

Người nói: Ngo Anh Duc **Mốc thời gian:** Trình bày 13:40–14:35

Lời thoại

Để đưa tới doanh nghiệp, còn ba nhóm việc phải làm. Nhóm đầu là dữ liệu thật có sự đồng ý và nhận review đủ tin cậy để đo FMR, FNMR và fairness trên cohort phù hợp. Nhóm thứ hai là tải và vận hành: năm mươi nghìn nhân viên không đồng nghĩa năm mươi nghìn request đồng thời, nên phải đo theo lịch sử dụng thực tế và bổ sung các điều kiện triển khai. Nhóm thứ ba là những điểm chưa khép kín trong code hiện tại: lifecycle mới tạo báo cáo RAI nhưng chưa áp dụng cờ fairness legacy làm gate, và registry mới chưa có quy trình tự tạo champion production đã đánh giá. Nhóm ghi rõ những giới hạn đó để phân biệt một demo MLOps có kiểm chứng với một sản phẩm đã sẵn sàng triển khai đại trà.

Thao tác / chuyển lời

Nêu giới hạn dứt khoát, sau đó chuyển sang trách nhiệm hoàn thiện của nhóm.

Nguồn đối chiếu: ARCHITECTURE.md; README.md; SCALABILITY_COST.md

Slide 18: Xác minh danh tính và kiểm soát thay đổi model

Người nói: Ngo Anh Duc **Mốc thời gian:** Trình bày 14:35–15:00

Lời thoại

Phần trình bày vừa rồi nối bài toán thi hộ với cách xác minh danh tính và kiểm soát thay đổi của model. Dịch vụ trả tín hiệu và bằng chứng để doanh nghiệp xem xét, còn model mới phải qua các gate trước khi phục vụ. Sau đây nhóm dành khoảng mười ba phút để đi trọn luồng sản phẩm, monitoring và hai kịch bản simulation. Hải điều khiển máy, Đức dẫn demo, Dương và Hiệp giải thích các điểm kỹ thuật khi cần.

Thao tác / chuyển lời

Chuyển sang phần demo. Khởi động đồng hồ demo riêng.

Nguồn đối chiếu: README.md; docs/MODALITY_LIFECYCLE.md

Slide 19: Demo sản phẩm: tạo nhân viên và ghi danh

Người nói: Ngo Anh Duc dẫn, To Thanh Hai thao tác **Mốc thời gian:** Demo 00:00–02:00

Lời thoại

Đầu tiên nhóm vào không gian công ty A bằng operator session đã đăng nhập trước. Hải tạo một nhân viên có mã riêng cho buổi demo rồi mở trang ghi danh. Nhóm dùng hai ảnh và hai WAV được phép sử dụng. Sau khi ghi danh, trạng thái nhân viên phải là đủ mẫu. Nếu muốn minh họa self-enrollment, có thể dùng lời mời một lần trong 24 giờ, nhưng chỉ chọn một đường ghi danh trong buổi bảo vệ. Việc ghi danh là bắt đầu của luồng, chưa chứng minh danh tính pháp lý của người đăng ký. Mẫu đã chuẩn bị giúp tập trung vào cơ chế, còn capture camera và microphone cần được kiểm thử trước trên chính trình duyệt này.

Thao tác / chuyển lời

Demo 00:00–02:00. Không đăng ký công ty hoặc cấp key mới trên màn chiếu. Nếu inference chậm, dùng nhân viên dự phòng đã ghi danh và nói rõ.

Nguồn đối chiếu: ui/app.py; docs/EMPLOYEE_DEMO.md; docs/SIMULATION.md

Slide 20: Demo xác minh: đúng người, nghi vấn và bằng chứng

Người nói: Ngo Anh Duc dẫn, To Thanh Hai thao tác **Mốc thời gian:** Demo 02:00–05:00

Lời thoại

Ở trang Kiểm tra tích hợp, nhóm chọn người A và gửi ảnh cùng giọng nói mới của A. Chúng ta đọc kết quả thực tế, gồm scores, match, capability và reason. Nếu detector trả suspicious hoặc inconclusive vì capture không phù hợp, nhóm giải thích đúng trạng thái đó. Tiếp theo vẫn khai báo người A nhưng gửi media của B đã được phép sử dụng. Kịch bản này kiểm tra identity mismatch. Nhóm mở Lịch sử và báo cáo, chọn đúng phiên demo và check vừa tạo để xem lý do và evidence. Ở API và Webhook, nhóm kiểm tra delivery của check đó. Công ty B chỉ thấy roster của B. Nếu cần chứng minh chặn truy cập trực tiếp, dùng request cross-tenant đã chuẩn bị và xem HTTP 404. Các check của phần này được lưu vào tenant demo của ứng dụng chính, khác với vùng dữ liệu cách ly của simulation.

Thao tác / chuyển lời

Demo 02:00–05:00. Tránh gửi lại cùng media nhiều lần trong một phiên rồi gọi capture_reused là model lỗi. Không lưu human labels cho dữ liệu tổng hợp.

Nguồn đối chiếu: ui/app.py; docs/EMPLOYEE_DEMO.md; docs/SIMULATION.md

Slide 21: Demo vận hành: monitoring và khởi động simulation

Người nói: Ngo Anh Duc dẫn, To Thanh Hai thao tác **Mốc thời gian:** Demo 05:00–06:00

Lời thoại

Nhóm chuyển từ không gian công ty sang monitoring của nền tảng. Grafana thể hiện hoạt động dịch vụ và độ mới của các collector. Thiếu human labels có thể tạo trạng thái chờ. No data không có nghĩa là không có drift. Sau đó mở tab platform riêng, chọn Voice trong Simulation MLOps và bắt đầu kịch bản promotion. Đây là điểm chuyển ranh giới dữ liệu: từ đây, các vector và nhãn là synthetic, được lưu trong database và MLflow riêng. Airflow cần thời gian nhận job, nên nhóm dùng khoảng chờ để chỉ run ID và đường đi của alert.

Thao tác / chuyển lời

Demo 05:00–06:00. Dùng tab/browser profile riêng đã đăng nhập platform để giữ phiên operator.

Nguồn đối chiếu: ui/app.py; docs/EMPLOYEE_DEMO.md; docs/SIMULATION.md

Slide 22: Demo promotion: alert, các gate và champion

Người nói: Ngo Anh Duc, thao tác: To Thanh Hai **Mốc thời gian:** Demo 06:00–09:00

Lời thoại

Run promotion đã bắt đầu ở bước trước. Bây giờ nhóm theo dõi receipt của alert qua Prometheus và Alertmanager trước training. Tiếp theo mở phần offline để thấy candidate và champion được so trên cùng holdout. Shadow phải giữ champion làm câu trả lời, còn canary có số request dùng challenger tăng qua các stage. Trên MLflow simulation, kiểm tra model name có tiền tố simulation và version đang được alias champion trở tới. Kết thúc cần đọc kết quả của đúng run đang chiếu. Nếu chưa hoàn tất sau khung chờ, nhóm nói rõ và dùng slide evidence lịch sử tiếp theo để giữ thời gian. Những request trong simulation đi qua HTTP endpoint embedding, không phải ảnh hoặc WAV từ nhân viên thật.

Thao tác / chuyển lời

Demo 06:00–09:00. Run đã được tạo ở slide trước, không bấm nút 1 lần nữa. Xem alert receipt, offline, shadow, canary và version. Nếu chậm, dùng evidence lịch sử có ghi ngày.

Nguồn đối chiếu: ui/simulation.py; docs/SIMULATION.md; VERIFICATION.md

Slide 23: Lướt promotion đã ghi nhận có traffic thực qua từng stage

Người nói: Ngo Anh Duc **Mốc thời gian:** Demo 09:00–09:30

Lời thoại

Đây là lướt promotion đã được lưu ngày 3 tháng 10, không phải số đếm được bịa để minh họa. Với 200 request mỗi stage, số request dùng challenger tăng theo rollout. Tỷ lệ thực tế ở stage nhỏ không nhất thiết đúng tuyệt đối 5 hay 10 phần trăm vì đây là mẫu hữu hạn được phân tuyến bằng hash. Shadow có không request nào dùng challenger để trả kết quả. Sau promotion, 200 probe đều dùng version 2. Đây là HTTP traffic tới endpoint embedding của simulation, không phải 200 nhân viên đã thực hiện capture thật.

Thao tác / chuyển lời

Demo 09:00–09:30. Nếu run live đã xong, ưu tiên số của run live. Bảng trên slide là evidence lịch sử ngày 03/10, không thay thế kết quả của run mới.

Nguồn đối chiếu: VERIFICATION.md: Simulation cách ly 03/10/2026; reports/simulation-promotion.json

Slide 24: Demo 2: lỗi canary phải dừng rollout

Người nói: Ngo Anh Duc, thao tác: To Thanh Hai **Mốc thời gian:** Demo 09:30–12:00

Lời thoại

Trước kịch bản thứ hai, nhóm khôi phục baseline demo. Việc reset chỉ áp dụng trong vùng simulation, không xóa dữ liệu production hoặc lịch sử audit. Lần này chọn Face và bấm Canary lỗi, rollback. Candidate vẫn đi qua calibration, offline và shadow. Điểm khác là tại canary 25 phần trăm, kịch bản đưa vào impostor có score cao để làm FMR vượt điều kiện an toàn. Điều nhóm cần chứng minh không phải job lúc nào cũng xanh theo nghĩa model được promote. Điều cần chứng minh là gate dừng rollout, traffic quay lại champion và nguyên nhân thất bại được lưu. Trạng thái FAILED_CANARY ở đây là kết quả đúng của kịch bản. Nếu đang chờ scheduler, nhóm chuyển sang evidence đã lưu ở slide tiếp theo, nói rõ đây là bằng chứng lịch sử. Sau khi kiểm tra, khôi phục baseline một lần nữa để kết thúc demo.

Thao tác / chuyển lời

Demo 09:30–12:00. Reset baseline, chọn Face, bấm nút 2 một lần. Kiểm tra failure tại 25% và traffic quay về champion. Nếu hạ tầng chậm, chuyển sang evidence lịch sử.

Nguồn đối chiếu: docs/SIMULATION.md; ui/simulation.py; reports/simulation-rollback.json

Slide 25: Rollback giữ lại model ổn định và bằng chứng thất bại

Người nói: Ngo Anh Duc **Mốc thời gian:** Demo 12:00–12:30

Lời thoại

Trong lượt rollback đã ghi nhận, failure xảy ra tại canary 25 phần trăm. Các mẫu impostor được tiêm làm FMR challenger vượt mức không hồi quy của champion. Sau khi dừng rollout, cả 200 probe đều dùng version 1. Model thất bại không bị xóa, vì nhóm vẫn cần lý do, stage, metric và phiên bản để điều tra. Báo cáo đối chiếu cũng xác nhận policy production không thay đổi qua hai kịch bản simulation. Đây là điều kiện để demo lỗi trên lớp mà vẫn giữ môi trường vận hành ổn định.

Thao tác / chuyển lời

Demo 12:00–12:30. Đối chiếu probe và failed metric trong run live. Bảng tóm tắt trên slide là lượt lịch sử.

Nguồn đối chiếu: reports/simulation-verification.json; VERIFICATION.md: Simulation cách ly 03/10/2026

Slide 26: Kết thúc demo: phục hồi baseline và giữ audit

Người nói: Ngo Anh Duc dẫn, To Thanh Hai thao tác **Mốc thời gian:** Demo 12:30–13:00

Lời thoại

Nhóm kết thúc bằng nút khôi phục baseline demo và xác nhận trạng thái reset. Lịch sử cùng evidence vẫn được giữ để người khác kiểm tra. Nếu có snapshot production policy trước và sau, nhóm đối chiếu champion, traffic và lifecycle state để chứng minh hai simulation không thay đổi chúng. Các check của phần demo sản phẩm vẫn là dữ liệu đã thêm vào tenant demo, không bị xóa bởi nút reset simulation. Nhóm đã trình diễn cả đường thành công, đường lỗi và cách đưa môi trường demo về trạng thái ban đầu. Phần còn lại là mười phút trao đổi với thầy cô và các bạn.

Thao tác / chuyển lời

Demo 12:30–13:00. Dừng thao tác tạo dữ liệu, chuyển slide Q&A. Nếu cancel còn chờ thì nói rõ, không báo đã reset thành công.

Nguồn đối chiếu: ui/app.py; docs/EMPLOYEE_DEMO.md; docs/SIMULATION.md

Slide 27: Trao đổi và phản biện

Người nói: Cả nhóm, Trịnh Đức Dương điều phối **Mốc thời gian:** Q&A 00:00–10:00

Lời thoại

Mời thầy cô và các bạn đặt câu hỏi. Dương nhắc lại câu hỏi ngắn gọn, chuyển cho người phụ trách rồi bổ sung nếu cần. Hiệp trả lời drift, evaluation và nhấn. Hải trả lời tích hợp, CI/CD và vận hành. Đức mở đúng slide phụ lục hoặc evidence. Mỗi câu hỏi thường trả lời trong khoảng một phút đến một phút rưỡi để dành thời gian cho câu hỏi tiếp theo. Nếu chưa có câu hỏi, dùng câu hỏi dự phòng trong QA_GUIDE.md, không tự hỏi và đọc hết cả danh sách.

Thao tác / chuyển lời

Bắt đầu đồng hồ Q&A riêng. Khi còn một phút, nhận câu cuối và kết luận.

Nguồn đối chiếu: CONTRIBUTING.md; ARCHITECTURE.md

Slide 28: Phụ lục: công thức và dữ liệu đầu vào

Người nói: Do Quang Hiep **Mốc thời gian:** Mở theo câu hỏi, không tính thêm thời gian

Lời thoại

PSI so sánh tỷ lệ quan sát trong các bin đã xác định từ reference. Nó được dùng cho feature chất lượng và score scalar. MMD sử dụng khoảng cách giữa các vector chuẩn hóa thông qua kernel RBF để đánh giá phân phối đa chiều. FMR và FNMR chỉ lấy nhãn reviewed đáng tin cậy, không lấy quyết định model làm nhãn. EER là ước lượng từ điểm giao ROC thực nghiệm, còn TAR tại FAR là TAR tốt nhất thỏa budget trên tập quan sát. Mẫu hữu hạn không đủ để suy ra bảo đảm FAR cho toàn bộ dân số.

Thao tác / chuyển lời

Chỉ mở khi cần giải thích phương pháp check drift hoặc input/output.

Nguồn đối chiếu: pipeline/drift_decision.py

Slide 29: Phụ lục: cấu hình thật và cấu hình simulation

Người nói: Do Quang Hiep **Mốc thời gian:** Mở theo câu hỏi, không tính thêm thời gian

Lời thoại

Simulation rút ngắn thời gian và số mẫu để phù hợp demo trên lớp, đồng thời cho phép mức disagreement cao hơn vì kịch bản hiệu chỉnh ngưỡng được dựng khá lớn. Nó không sửa file cấu hình production. Các stage traffic vẫn là 5, 10, 25, 50 và 100 phần trăm. Những giá trị mặc định production trong bảng cũng là pilot settings cần hiệu chỉnh bằng dữ liệu khách hàng, không phải tiêu chuẩn an toàn đã được chứng minh cho mọi tổ chức.

Thao tác / chuyển lời

Dùng để trả lời vì sao demo chỉ chạy vài phút trong khi stage thật cần một giờ.

Nguồn đối chiếu: pipeline/lifecycle_config.json; docs/SIMULATION.md

Slide 30: Phụ lục: bốn câu hỏi dễ bị hiểu nhầm

Người nói: Trịnh Đức Dương **Mốc thời gian:** Mở theo câu hỏi, không tính thêm thời gian

Lời thoại

Khi trả lời, luôn nêu đúng đối tượng. Từ model trong lifecycle này chỉ threshold policy, không phải mọi thành phần pretrained. Từ canary chỉ policy routing trong API, không phải nhiều phiên bản container. Từ simulation nói tới dữ liệu tổng hợp nhưng có các thao tác HTTP, alert và Registry thực trong môi trường riêng. Một lần demo thành công chỉ chứng minh luồng đó chạy với dữ liệu đó. Nếu được hỏi model tốt nhất, giải thích champion là policy được chấp nhận theo gate và bằng chứng hiện có, không phải tối ưu tuyệt đối cho mọi phân phối tương lai.

Thao tác / chuyển lời

Có thể dùng trang này như tờ nhắc trước khi bảo vệ.

Nguồn đối chiếu: ARCHITECTURE.md; docs/SIMULATION.md; docs/MODALITY_LIFECYCLE.md

Slide 31: Bằng chứng hiện có tập trung vào luồng và tính an toàn

Người nói: Ngo Anh Duc **Mốc thời gian:** Mở theo câu hỏi, không tính thêm thời gian

Lời thoại

Nhóm phân loại bằng chứng thay vì gộp tất cả thành một con số accuracy. Lần kiểm chứng local được ghi nhận ngày 3 tháng 10 có 153 test pass, không skip, với coverage 82 phần trăm trong phạm vi khai báo. Ngoài unit test, hai kịch bản simulation đã chạy qua Airflow, nhận alert thật, dùng HTTP traffic và cập nhật MLflow riêng. Những bằng chứng này cho thấy cơ chế vận hành và các gate có thể được kiểm tra. Chúng không chứng minh tỷ lệ phát hiện gian lận trên nhân viên thật. Tiếp theo nhóm sẽ minh họa đúng hai kịch bản đó, với dữ liệu được gắn nhãn tổng hợp rõ ràng.

Thao tác / chuyển lời

Nếu được hỏi CI mới nhất, mở đúng run thực tế; không suy từ số test local thành trạng thái CI hiện tại.

Nguồn đối chiếu: VERIFICATION.md: Simulation cách ly 03/10/2026; reports/simulation-verification.json

Slide 32: Trách nhiệm được chia theo các phần của hệ thống

Người nói: Ngo Anh Duc **Mốc thời gian:** Mở theo câu hỏi, không tính thêm thời gian

Lời thoại

Đây là phân công trách nhiệm để nhóm duy trì và bàn giao dự án. Dương phụ trách core service và vòng đời model. Hiệp phụ trách kiểm chứng chất lượng, evaluation và evidence. Hải phụ trách container, runner và triển khai. Đức phụ trách báo cáo, sơ đồ, slide và tổ chức demo. Việc phân công này cần được chứng minh bằng phần việc và review thực tế; nhóm không dùng riêng metadata Git để thay thế bằng chứng đóng góp. Khi bảo vệ, mỗi thành viên cần giải thích được phần mình và mối nối với toàn bộ hệ thống.

Thao tác / chuyển lời

Không đọc email hoặc đưa lịch sử rewrite Git lên slide trình bày chính.

Nguồn đối chiếu: CONTRIBUTING.md

Slide 33: Phụ lục: nguồn và cách đối chiếu bằng chứng

Người nói: Ngo Anh Duc **Mốc thời gian:** Mở theo câu hỏi, không tính thêm thời gian

Lời thoại

Nguồn của bộ slide là implementation và tài liệu hiện có trong repository. Evidence simulation được đối chiếu với báo cáo đã lưu, nhưng bản thuyết trình chỉ đưa số liệu tổng hợp, không chứa credentials hoặc biometric media. Các số liệu có ngày kiểm chứng để tránh nhầm với trạng thái runtime hiện tại. Nếu cần công bố accuracy hoặc tác động giảm gian lận, nhóm phải thực hiện nghiên cứu riêng trên dữ liệu được phép sử dụng. Không có tuyên bố đó trong bộ slide này.

Thao tác / chuyển lời

Repo công khai: <https://github.com/FSB-MSA36HN/DDM501-face-voice-proctoring>

Nguồn đối chiếu: <https://github.com/FSB-MSA36HN/DDM501-face-voice-proctoring>; VERIFICATION.md

Khi demo không diễn ra như dự kiến

Tình huống	Xử lý và câu nói
Queue chờ hơn một phút	Kiểm tra scheduler và DAG. Nói: "Job đang chờ Airflow nhận, đây chưa phải kết quả gate."
Chưa nhận alert	Xem Prometheus/Alertmanager và alert receipt. Không bỏ gate chờ alert hoặc bấm lại liên tục.
Promotion chưa xong trong khung giờ	Nói: "Lượt hiện tại còn chạy. Em chuyển sang evidence đã lưu ngày 03/10 để giữ thời lượng." Mở slide 23.
FAILED_CANARY trong scenario rollback	Đây là kết quả mong đợi. Chỉ failure metric, traffic zero và probe dùng champion.
FAILED do hạ tầng	Nói rõ đây là lỗi hạ tầng, không đổi tên thành rollback thành công. Dùng evidence lịch sử rồi reset khi bước active dùng.
Reset đang chờ	Đợi bước đang chạy kết thúc/cancel. Không xóa DB, container volume hoặc giả định trạng thái đã reset.

Câu hỏi dự kiến và câu trả lời ngắn

Tại sao phải dùng cả Face và Voice? Hai modality cung cấp bằng chứng danh tính khác nhau cho bối cảnh có ảnh và phần nói. Nhóm chưa có benchmark để khẳng định một tỷ lệ cải thiện accuracy cụ thể so với chỉ một modality.

Có bắt được mọi cheating không? Không. Phạm vi là danh tính và tín hiệu integrity trên các capture được gửi. Đọc tài liệu ngoài màn hình, gợi ý bên ngoài và mọi kiểu deepfake không được bảo đảm phát hiện.

Champion có phải model tốt nhất không? Là policy được chấp nhận theo gate và bằng chứng hiện có để phục vụ. Không phải tối ưu tuyệt đối trên mọi dữ liệu tương lai. Challenger vẫn cần offline, shadow và canary.

Vì sao score của champion và challenger bằng nhau? Encoder và cosine được dùng chung. Policy hiện khác ở threshold, nên score bằng nhau nhưng decision có thể khác.

Data drift có tự retrain không? Một tín hiệu thống kê riêng lẻ không đủ. Cần persistence, trusted performance degradation, đủ dữ liệu mới, nhiều cohort và không chủ yếu do input/template aging.

Embedding được lưu ở đâu? PostgreSQL dạng JSON. Đây là verification 1:1 theo employee đã khai báo, không phải nearest-neighbor search toàn bộ nhân viên.

Thiếu nhãn thì làm gì? Trả INSUFFICIENT_DATA hoặc dùng progression để thu review phù hợp. Không dùng accept/reject của model làm ground truth.

Simulation có ảnh hưởng hệ thống thật không? Dữ liệu, DB và MLflow riêng; không đổi production policies. Host và Airflow dùng chung nên có thể ảnh hưởng tài nguyên/latency. Không khẳng định cách ly phần cứng.

Demo canary thành công chứng minh điều gì? Chứng minh routing, gates và Registry transitions chạy trên dữ liệu synthetic. Không chứng minh mức FAR/FNMR trong dân số hoặc accuracy trên người thật.

Tại sao không dùng ngưỡng 20% trong báo cáo cũ? Đó là gate của bundle demo lịch sử. Lifecycle mới dùng cấu hình per-modality tại `pipeline/lifecycle_config.json`; cần gắn mọi con số với đúng phiên bản.

50.000 nhân viên đã chạy được chưa? Chưa có load/concurrency benchmark ở quy mô đó. Cần xác định lịch thi, số request đồng thời, sizing và retention trước pilot.

Ai quyết định nhân viên gian lận? Dịch vụ trả tín hiệu, lý do và bằng chứng. Khách hàng quyết định nghiệp vụ theo quy trình review, không tự động kỷ luật từ nhãn suspicious.

Cách rút ngắn nếu gần hết giờ

Giữ câu chuyện, sơ đồ I/O, decision engine, lifecycle và một kịch bản demo. Rút phần bảng feature/kiến trúc chi tiết, dùng evidence lịch sử thay cho kịch bản live thứ hai. Không cắt câu phân biệt dữ liệu tổng hợp với benchmark thật hoặc phần giới hạn của dự án.

Kịch bản demo toàn diện: 10–15 phút

Vị trí trong buổi bảo vệ: sau 15 phút trình bày, trước 10 phút Q&A. **Bản chuẩn:** 13 phút, slide 19–26.

Người dẫn: Đức. **Thao tác:** Hải. Dương giải thích lifecycle; Hiệp đọc metric/gate và kiểm chứng kết quả.

Mục tiêu là đi từ pain point “ai thực sự làm bài?” tới bằng chứng xác minh, rồi chứng minh hệ thống biết kiểm soát việc cập nhật policy. Không cố chứng minh mọi loại cheating trong một buổi demo.

1. Ranh giới dữ liệu cần nói trước

- **Demo sản phẩm:** tạo nhân viên/checks/callbacks/evidence trong tenant demo trên ứng dụng chính. Những thao tác này có ghi dữ liệu. Dùng tenant riêng của buổi học, không dùng tenant khách hàng. Reset simulation không xóa dữ liệu phần này.
- **Demo simulation:** dùng vector/nhân tổng hợp, SQLite/MLflow/volume riêng, không thay đổi production policy. Dùng chung host và Airflow nên vẫn có thể tranh chấp CPU/RAM. Không nói “không có bất kỳ tác động nào tới production”.
- **Evidence lịch sử:** số liệu trên slide 23/25 là lượt ngày 03/10/2026. Nếu mở report lịch sử, nói rõ là report đã lưu. Run mới phải được đánh giá bằng run ID mới.

2. Chuẩn bị trước giờ học

Thực hiện trước khi chia sẻ màn hình. Không đưa credentials hoặc biometric media lên GitHub. Đây là checklist chuẩn bị, không có nghĩa các bước đã được chạy lại trong lần biên soạn tài liệu này.

Trước 30–60 phút

1. Kiểm tra Docker Desktop, API, portal, Airflow scheduler, Grafana và hai MLflow services hoạt động. Runtime phải có champion đã đánh giá và data/artifacts tương ứng nếu demo luồng sản phẩm. Registry trống có thể `/health = 200` nhưng `/ready = 503`; không chữa bằng cách tùy tiện đổi alias hoặc hạ gate.
2. Chuẩn bị hai tenant **DEMO-CLASS-A**, **DEMO-CLASS-B** từ trước. Đăng nhập operator A, operator B và platform bằng ba browser profile/session riêng. Hai tab trong cùng profile có thể chia sẻ trạng thái; kiểm tra lại tên tenant trên giao diện. Gói đăng ký chỉ là giả lập, không có thanh toán thật.
3. Chuẩn bị người A và người B với media có quyền sử dụng. Mỗi người có hai ảnh và hai WAV ghi danh khác nhau; thêm một cặp ảnh/WAV để query. Audio khoảng 10 giây phù hợp phần nói, không dùng WAV quá ngắn rồi coi capability unavailable là lỗi.
4. Test trước media trên cùng runtime. Ghi lại expected identity match, inspector capabilities và reasons thực tế. Không cam kết “đúng người luôn verified” nếu detector anti-spoof hoặc chất lượng có thể trả suspicious/inconclusive.
5. Có nhân viên **A-READY** đã ghi danh để dự phòng và một mã mới, ví dụ **CLASS-YYYYMMDD-01**, để tạo live. Không ghi danh lại cùng file vào một người đã có mẫu rồi gọi lỗi duplicate là bug.
6. Dùng bộ media query mới cho mỗi lượt trong một session. Nếu dùng lại file trong lần tập dượt, đổi session sang **CLASS-YYYYMMDD-HHMM**. Không dùng thủ thuật đó để phủ nhận tín hiệu capture reuse; nói rõ đây là cách tách hai phiên demo.

7. Cấu hình webhook trước giờ: receiver thật được allowlist và sẵn sàng ACK. Receiver demo tại <http://legacy-demo:8000/webhooks/verification> phải nhận biết đúng tenant và signing secret của tenant đó. Chỉ đặt URL chưa đủ để đảm bảo delivery. Tập thử và kiểm tra **delivered**, HTTP 200 trước buổi học.
8. Chuẩn bị tùy chọn request trong API client với secrets nằm trong biến môi trường cục bộ: retry cùng payload và cross-tenant GET. Không chiếu request headers, terminal history hoặc environment có key. Chỉ mở response/status đã lọc.
9. Chạy thử cả hai scenario simulation một lần, xuất JSON và ghi lại thời gian. Dùng nút **3. Khôi phục baseline demo**, xác nhận reset đã hoàn tất. Không reset bằng `docker compose down -v`, xóa database hoặc xóa model.
10. Chuẩn bị report đã lọc của promotion/rollback và screenshot kết quả của chính lần tập dượt để dự phòng. Giữ timestamp/run ID, không sửa số liệu để khớp slide.

Các cửa sổ mở sẵn

Cửa sổ	URL / nơi mở	Nội dung cần thấy
Operator công ty A	http://localhost:18501	Nhân viên, ghi danh, checks, lịch sử
Operator công ty B	Profile riêng, cùng URL	Roster/lịch sử của B
Platform	Profile riêng, cùng URL	Simulation MLOps
Nhân viên, tùy chọn	http://localhost:18600	Self-enrollment qua lời mời dùng một lần
Grafana	http://localhost:13000/d/biometric-overview	Service metrics, freshness, lifecycle
Airflow	http://localhost:18081/dags/biometric_simulation/grid	Run của chính simulation đang demo
MLflow simulation	http://localhost:15031	Model simulation- <code><run-id></code> - <code><modality></code>
MLflow production	http://localhost:15030	Face/Voice registry độc lập, chỉ đọc
GitHub Actions FSB	https://github.com/FSB-MSA36HN/DDM501-face-voice-proctoring/actions	Run cụ thể, job và artifacts

Mở GitHub FSB trực tiếp hoặc link CI/CD trong trang Vận hành MLOps. Trong buổi demo cũng không dùng nút **Reload champion model** để thay cho lifecycle rollback, vì đó là thao tác bundle legacy.

3. Timeline chuẩn 13 phút

Thời gian demo	Slide	Thao tác chính	Điểm cần chứng minh
00:00–02:00	19	Tạo nhân viên, ghi danh	Mẫu và identity được gắn với đúng tenant/người
02:00–05:00	20	Same-person, other-person, history/evidence/webhook	Kết quả có reason, version và bằng chứng truy xuất
05:00–06:00	21	Grafana, bắt đầu simulation Voice	Tách monitoring nghiệp vụ và nền tảng
06:00–09:00	22	Theo dõi promotion	Alert, offline, shadow, canary và champion
09:00–09:30	23	Đọc routed counts	Phân biệt traffic live với bằng evidence lịch sử
09:30–12:00	24	Reset, scenario Face rollback	Fail gate dừng rollout
12:00–12:30	25	Kiểm tra probes/version	Traffic thực quay về champion
12:30–13:00	26	Reset, lưu evidence, chuyển Q&A	Kết thúc có trạng thái và audit rõ ràng

Bước A. Ghi danh: 00:00–02:00

Đức nói: “Em mở công ty demo A. Phần này minh họa dữ liệu thực sự được ghi vào tenant demo của ứng dụng. Chúng ta tạo một nhân viên và gắn mẫu Face/Voice với người đó trước khi có thể xác minh.”

Hải thao tác:

- Trên operator A, vào **Nhân viên**, nhập mã mới và tên hiển thị, bấm **Thêm nhân viên**.
- Vào **Ghi danh face & voice**, chọn đúng người. Upload đúng hai ảnh và hai WAV đã chuẩn bị; checkbox consent chỉ chọn khi thực sự có sự đồng ý/quyền sử dụng.
- Bấm **Ghi danh** một lần. Đọc kết quả và kiểm tra **Đủ mẫu** trong danh sách.
- Nếu muốn self-enrollment, thay bước 2–3 bằng **Tạo liên kết ghi danh**, mở :18600, **Kiểm tra lời mời**, gửi bốn mẫu. Chọn một đường, không demo cả hai trong bản 13 phút.

Kết quả cần đọc: nhân viên đúng tenant, **ready=true** hoặc nhấn **Đủ mẫu**. Nếu thiếu mẫu, đọc **rejected** và lý do. Sau 30 giây điều tra không xong, chuyển sang A-READY và nói rõ đó là nhân viên đã chuẩn bị trước, không giả vờ lượt enrollment live thành công.

Lời nói: “Chúng ta đã có reference của người A. Bây giờ xem điều gì xảy ra khi capture đúng người và khi tài khoản A nhận media của người khác.”

Bước B. Xác minh và bằng chứng: 02:00–05:00

- Kiểm tra tích hợp:** chọn A, nhập session mới, upload query của A, xác nhận consent và bấm **Gửi check**. UI sinh request ID mới mỗi lần bấm.

2. Mở JSON kỹ thuật vừa đủ để chỉ `check_id`, face/voice scores và match, `reason_codes`, `capabilities`, `model_versions` / `model_version` và `evidence_status`. Không đọc toàn JSON.
3. Giữ khai báo người A, gửi ảnh/WAV của B đã được phép dùng. Kỳ vọng identity mismatch trong bộ fixture đã tập thử. Chỉ nói kết quả thật đang thấy; không tuyên bố anti-spoof accuracy từ một ví dụ.
4. **Lịch sử & báo cáo**: lọc người A và session vừa nhập, chọn check nghi vấn, bấm **Mở bằng chứng** khi `stored` hoặc `partial`. Chỉ phát 2–3 giây audio nếu phù hợp quyền sử dụng. Xuất CSV hoặc PDF, không cần xuất cả hai trong bản chuẩn.
5. **API & Webhook**: xem bảng delivery của check vừa gửi, chỉ status/HTTP code. Không chọn **Hiển thị secret xác thực webhook**, không cấp hoặc thu hồi key live.
6. Nếu còn 20–30 giây, chuyển operator B để thấy dữ liệu B. Đây là bằng chứng giao diện được scope, chưa đủ chứng minh chặn direct API. Muốn chứng minh mạnh hơn, dùng request đã chuẩn bị `GET /v1/checks/{check_A}` với key B, response 404.

Đức nói: “Một score thấp chưa phải kết luận gian lận. Chúng ta xem identity, capture quality, capability và reason trước. Lịch sử chỉ thể hiện các capture đã nhận, không chứng minh giám sát liên tục ở khoảng giữa hai lượt.”

Hiệp bổ sung: “Review labels phải có căn cứ độc lập. Em chỉ mở form để chỉ ra Face/Voice labels riêng, không lưu dữ liệu synthetic thành human ground truth để kích hoạt retrain.”

Tùy chọn mở rộng 30–45 giây: trong API client đã chuẩn bị, gửi lại đúng `request_id` và payload để thấy cùng `check_id`. Đổi payload nhưng giữ request ID thì 409. **Bấm Gửi check hai lần trên UI không phải test idempotency**, vì mỗi lần UI tạo UUID mới và có thể kích hoạt capture reuse.

Bước C. Vận hành và chuyển ranh giới: 05:00–06:00

Mở Grafana: chỉ một panel service/latency, freshness và một panel lifecycle theo modality. Không cuộn qua toàn dashboard. `No data` do thiếu reviewed labels không có nghĩa performance khỏe. Sau đó sang platform **Simulation MLOps**, chọn Voice, ghi nhận champion simulation ban đầu và bấm **1. Drift → nâng cấp champion** một lần.

Đức nói: “Các check vừa rồi đi qua ứng dụng chính. Từ bước này, demo dùng vector và nhãn tổng hợp trong vùng simulation riêng. Chúng ta dùng dữ liệu được dựng để kiểm tra cơ chế xử lý drift, không dùng nó để công bố accuracy.”

Trong lúc Airflow chờ nhận job, có thể mở GitHub FSB run đã chọn sẵn tối đa 20 giây: chỉ Windows `deployment-preflight`, build và `deploy-demo`, phân biệt preflight với deploy thật. Chỉ gọi job success khi run thực tế hiển thị success. Không bấm rerun hoặc deploy để phục vụ màn trình diễn.

Bước D. Promotion: 06:00–09:30

Không tạo run thứ hai: run đã bắt đầu ở bước C. Đối chiếu cùng run ID giữa portal và Airflow. Mở **Quyết định drift, gate và lịch sử chuyển trạng thái**.

Quan sát	Ý nghĩa cần nói	Tiêu chí đọc kết quả
Drift report	Ba cửa sổ mới, score/embedding drift và reviewed degradation	RETRAIN_REQUIRED của modality được chọn
Alert receipt	Prometheus/Alertmanager đã giao alert	Receipt có đúng run ID trước training
Offline	So cùng holdout	FMR/FNMR candidate không vi phạm gate
Shadow	Challenger quan sát, champion trả lời	Served candidate = 0 ở shadow
Canary	Request thật dùng policy theo cohort	Stage và served_candidate tăng theo traffic
Promotion	Chỉ sau stage cuối	Version champion mới, probe đúng version

Dương nói: “Training ở đây chọn threshold bằng calibration. Candidate phải được so với incumbent trên cùng dữ liệu holdout. Trong shadow, đáp án người dùng vẫn thuộc champion. Tới canary mới có request dùng challenger, nên cần nhìn routed counts, không chỉ nhìn phần trăm cấu hình.”

Mở MLflow :15031, chọn model có đúng run ID/modality. Đọc **champion** và version; không mở :15030 rồi đổi production aliases. Chuẩn bị tải evidence JSON cho đúng run sau khi hoàn tất. Nếu tiếp tục chạy tới quá demo 09:00, dùng slide 23 và báo rõ đó là evidence lịch sử. Muốn bắt đầu scenario rollback phải reset/cancel run trước và đợi reset hoàn tất; nếu không đủ thời gian, dùng evidence lịch sử cho rollback.

Bảng slide 23 ngày 03/10: shadow 0/200, canary 13/22/57/99/200 trên 200 request mỗi stage. Đây là một lượt cụ thể, không phải số bắt buộc mọi seed/run phải giống hệt.

Bước E. Rollback: 09:30–12:30

1. Bấm **3. Khôi phục baseline demo**, chờ xác nhận. Chọn Face.
2. Bấm **2. Canary lỗi → rollback** một lần. Theo dõi cùng run, không bấm lại khi đang chờ alert hoặc scheduler.
3. Đọc failure tại canary 25%, **failed metric**, observed value, limit, stage/version. Kịch bản chủ động tiêm impostor score cao; không gọi đây là lỗi tự phát đã phát hiện trên dữ liệu nhân viên thật.
4. Đối chiếu traffic về 0 và rollback probes dùng incumbent. Trong lượt lịch sử, 200/200 probe dùng version 1. Run mới đọc số thực tế của nó.

Hiệp nói: “Một candidate giảm từ chối nhằm vẫn có thể nguy hiểm nếu tăng chấp nhận sai. Gate FMR chặn trường hợp đó. Trạng thái FAILED_CANARY ở kịch bản này là kết quả đúng: challenger bị dừng, champion được giữ và evidence vẫn còn.”

Phân biệt: workflow hoàn tất kịch bản rollback không có nghĩa model được promote. Một exception do network/database cũng không phải bằng chứng gate rollback hoạt động.

Bước F. Reset và kết thúc: 12:30–13:00

1. Lưu report cho đúng hai run, dùng **Chuẩn bị báo cáo evidence JSON** rồi tải file.
2. Bấm **3. Khôi phục baseline demo**. Đợi UI xác nhận phục hồi baseline. Reports drift cũ vẫn còn là history; đừng gọi history đó là trạng thái drift mới.

3. Nếu đã chuẩn bị snapshot trước/sau `GET /v1/admin/lifecycle/state` với platform credential, đối chiếu **modality, state, traffic_percent, champion_version, challenger_version**. Không so toàn JSON byte-for-byte vì samples/timestamps có thể thay đổi. Các check của bước B có thể làm counters tăng.

4. Nếu chưa có snapshot, chỉ nói “thiết kế cách ly và evidence lịch sử ghi nhận production policies không đổi”, không khẳng định đã kiểm chứng trước/sau buổi live.

Đức kết: “Nhóm đã đi qua xác minh danh tính, xem lại bằng chứng, quan sát hệ thống, đưa policy mới qua gate và chủ động chặn một policy gây regression. Xin mời thầy cô và các bạn đặt câu hỏi trong mười phút tiếp theo.”

4. Điều chỉnh theo thời gian thực tế

Bản demo	Sản phẩm	Vận hành	Promotion	Rollback	Reset/kết
10 phút	3:00	0:45	3:00	2:45	0:30
13 phút chuẩn	5:00	1:00	3:30	3:00	0:30
15 phút tối đa	6:00	1:30	3:30	3:00	1:00

Bản 10 phút dùng A-READY để giới thiệu enrollment đã chuẩn bị, chỉ mở một export, và có thể dùng report lịch sử cho scenario thứ hai nếu chưa hoàn tất. Bản 15 phút thêm self-enrollment **hoặc** idempotency/cross-tenant direct API, không cố thêm tất cả. Hai kịch bản simulation chạy nối tiếp vì service chỉ nhận một run active.

5. Kịch bản xử lý sự cố

Sự cố	Hành động	Câu nói trung thực
Camera/mic không cấp quyền	Dùng upload được chuẩn bị và quyền sử dụng hợp lệ	“Em chuyển sang media fixture để tiếp tục kiểm tra luồng.”
Capture đúng người nhưng suspicious	Xem inspector reasons và chất lượng	“Identity và integrity là hai loại tín hiệu, đây là kết quả thực tế của capture này.”
Webhook pending / failed	Xem delivery status, không sửa secret trên màn chiếu	“Callback chưa được receiver xác nhận; em giữ đúng trạng thái này và dùng evidence trước để giải thích retry.”
Airflow queue chờ	Xem scheduler/run, dùng evidence lịch sử theo timebox	“Run hiện tại chưa hoàn tất, bảng tiếp theo thuộc lượt đã lưu.”
Alert chưa tới	Xem receipt, không bỏ điều kiện chờ hoặc hạ ngưỡng	“Training chưa được phép bắt đầu vì chưa có alert receipt đúng run.”
Reset đang chờ	Đợi bước active dừng, không xóa volumes	“Reset đang chờ bước đang chạy kết thúc.”
Model gate thật sự reject	Trình bày failure reason	“Candidate không đạt gate nên champion được giữ nguyên.”

6. Tiêu chí một buổi demo hoàn tất

- Có ít nhất một request sản phẩm live, đọc đúng kết quả và đúng tenant/session.
- Mở được lịch sử và evidence phù hợp hoặc nói rõ lý do chưa có evidence.
- Phân biệt alert nền tảng với business webhook của công ty.
- Với từng scenario, chỉ ra run ID, nguồn live/lịch sử, gate và outcome đúng.
- Đã yêu cầu reset simulation và xác nhận outcome thực tế, giữ audit.
- Không thay đổi gate, production alias hoặc gán synthetic data thành human labels.
- Không kết luận human accuracy, fairness hoặc capacity từ demo này.

Q&A 10 phút: câu hỏi và câu trả lời theo implementation

Dương điều phối, Hiệp trả lời evaluation/drift, Hải trả lời platform/integration, Đức mở evidence và giữ thời gian. Ưu tiên câu hỏi thật từ giảng viên. Các câu dưới là ngân hàng luyện tập, không đọc tuần tự toàn bộ trong 10 phút.

Cách phân bổ thời gian

Khoảng thời gian	Hoạt động
00:00–00:30	Mời câu hỏi, nhắc lại câu đầu nếu cần
00:30–08:30	Khoảng 5–6 câu hỏi, mỗi câu 60–90 giây kể cả follow-up
08:30–09:30	Câu cuối hoặc một câu chưa khép lại
09:30–10:00	Ghi nhận phần cần kiểm chứng thêm và cảm ơn

Mỗi câu trả lời theo ba nhịp: **trả lời trực tiếp, chỉ bằng chứng/code, nêu giới hạn liên quan**. Không tranh nhau nói. Người đầu trả lời xong mới mời người khác bổ sung. Nếu thiếu số liệu, nói “nhóm chưa có phép đo đó” thay vì dự đoán.

Nếu chưa có câu hỏi, gợi mở tối đa hai chủ đề: “Thầy cô muốn nhóm làm rõ việc thiếu nhãn thì xử lý drift thế nào, hay cách canary rollback giữ champion?” Không dùng Q&A để kéo dài bài trình bày thêm mười phút.

Các câu trọng tâm nên tập trước

1. Dự án có thực sự phát hiện gian lận không?

Người trả lời: Dương.

“Dự án phát hiện tín hiệu danh tính và integrity trên những capture đã nhận, như face/voice mismatch hoặc dấu hiệu media giả. Nó không đủ để kết luận mọi hành vi gian lận. Nhãn suspicious có reason/evidence để doanh nghiệp review; khách hàng giữ quyết định nghiệp vụ. Nhóm chưa đo tỷ lệ giảm cheating tại doanh nghiệp thật.”

Nếu hỏi sâu: Không bao phủ chắc chắn nhờ gợi ý ngoài màn hình, đọc tài liệu, mọi replay/deepfake hoặc khoảng thời gian giữa hai capture. Xem slide 4 và [SAAS_INTEGRATION.md](#) / [RESPONSIBLE_AI.md](#).

2. Model input/output là gì? Retrain cái gì?

Người trả lời: Dương, Hải bổ sung serving.

“Encoder nhận ảnh hoặc WAV, trả embedding chuẩn hóa. Serving tính max cosine với template của người được khai báo. Policy nhận score và áp dụng ngưỡng riêng Face hoặc Voice để đưa ra identity decision. Retrain hiện tại hiệu chỉnh threshold policy; SFace và ECAPA vẫn là pretrained encoder, chưa fine-tune weights.”

Nếu hỏi sâu: Inspector PAD/anti-spoof và quyết định integrity cuối là lớp riêng. Không gọi toàn bộ response API là output duy nhất của encoder. Xem slide 7, [api/app/biometrics.py](#), [pipeline/modality_training.py](#).

3. Check drift bằng phương pháp nào và theo feature nào?

Người trả lời: Hiệp.

"Quality và score dùng PSI. Quality gồm brightness, contrast, blur, face area hoặc audio RMS, clipping, silence, duration, sample rate theo những gì code đo được. Embedding drift dùng RBF MMD bình phương trên vector đa chiều chuẩn hóa. Performance dùng reviewed FMR/FNMR, EER và TAR@FAR. Cuối cùng so sai số theo tuổi template."

Nếu hỏi sâu: Không kiểm từng embedding dimension độc lập. SNR/yaw/occlusion không được tự đưa vào khi chưa có measurement. Mở slide 28, [pipeline/drift_decision.py](#).

4. Drift có lập tức trigger retrain không?

Người trả lời: Hiệp.

"Không. Quality drift đi sang kiểm tra capture. Embedding/score drift đơn lẻ là tín hiệu theo dõi. Retrain cần ba cửa sổ mới đủ điều kiện, performance có trusted labels suy giảm qua nhiều cohort tuổi, đủ dữ liệu mới và cooldown. Thiếu vector hoặc labels thì insufficient data, không tự train chỉ để pipeline tiếp tục."

Nếu hỏi sâu: Mặc định 100 observations/window, 20 nhãn/class, 40 reviewed mới, cooldown 24 giờ. Same-window polling không tăng persistence. Xem slide 12, [pipeline/lifecycle_config.json](#), [api/app/lifecycle_service.py](#).

5. Tại sao cập nhật template thay vì retrain model?

Người trả lời: Hiệp hoặc Dương.

"Nếu chỉ template cũ suy giảm trong khi template mới còn khỏe, encoder/policy có thể không phải nguyên nhân chính. Hệ thống tạo candidate template từ nhiều quan sát trusted, dùng holdout riêng và chỉ activate khi không tăng FMR hoặc FNMR. Giữ version trước để rollback. Một lần model accept không được coi là ground truth."

Nếu hỏi sâu: Đây là heuristic theo cohort, chưa phải chứng minh nhân quả. Thiếu bằng chứng ở PENDING_REVIEW; post-activation template rollback tự động chưa có. Xem slide 13, [api/app/template_lifecycle.py](#).

6. Champion/challenger là alias môi trường hay chọn model tốt hơn?

Người trả lời: Dương.

"Alias biểu thị vai trò của phiên bản trong cùng lifecycle. Champion là policy đang phục vụ; challenger là policy đã qua offline gate và đang được thử. Chúng không đơn thuần là dev/staging/prod. Chỉ sau shadow và canary mới chuyển champion. Champion được chấp nhận theo bằng chứng hiện có, không phải model tối ưu tuyệt đối."

Nếu hỏi sâu: Gate hiện chặn hồi quy và yêu cầu đạt budget, không nhất thiết bắt buộc candidate thắng tuyệt đối mọi metric. Face/Voice có registry name riêng. Xem slide 14, [api/app/model_lifecycle.py](#).

7. Score champion/challenger bằng nhau thì shadow có ý nghĩa gì?

Người trả lời: Dương.

"Hai policy dùng chung encoder và cosine nên score delta bằng không. Threshold khác nhau vẫn có thể làm quyết định khác nhau. Shadow ghi decision disagreement, reviewed FMR/FNMR và latency policy mà không ảnh hưởng response champion. Nó kiểm tra threshold rollout, không phải thử hai encoder khác nhau."

Nếu hỏi sâu: Policy p95 không phải latency end-to-end của inference/upload. Xem slide 7, 14 và [docs/MODALITY_LIFECYCLE.md](#).

8. Deployment có thật là canary không?

Người trả lời: Hải.

"Có canary ở lớp policy: hash ổn định theo modality/deployment/tenant/person/session chọn request dùng threshold challenger. Code ứng dụng vẫn deploy bằng thay container Docker Compose trên một host. Nhóm chưa triển khai application canary bằng nhiều replica hoặc Kubernetes."

Nếu hỏi sâu: Cần chỉ served_candidate count và response version, không chỉ traffic_percent. Xem slide 16, 23 và [api/app/model_lifecycle.py](#).

9. Demo simulation chứng minh được điều gì?

Người trả lời: Hiệp, Đức mở evidence.

"Nó chứng minh luồng drift, alert, calibration, offline gate, HTTP routing, Registry transition và rollback trên dữ liệu tổng hợp. Alert và request là thao tác thật trong môi trường simulation. Dữ liệu là vector/labels synthetic, không chạy camera hoặc WAV qua encoder, nên không chứng minh accuracy người thật."

Nếu hỏi sâu: Historical holdout được dựng để cho một threshold correction rõ ràng; FNMR giảm lớn không phải kết quả benchmark khách hàng. Xem [VERIFICATION.md](#) và [docs/SIMULATION.md](#).

10. Simulation có làm hỏng dữ liệu đang chạy không?

Người trả lời: Hải.

"Simulation có DB/MLflow/volume riêng và không nhận production credentials/weights. Reset chỉ phục hồi baseline simulation, giữ audit. Evidence lịch sử cho thấy policy production không đổi. Nhưng host/Airflow dùng chung nên vẫn có khả năng tranh chấp tài nguyên. Phần demo sản phẩm trước đó có thêm dữ liệu vào tenant demo của app."

Nếu hỏi sâu: Đối chiếu fields policy trước/sau nếu đã chụp snapshot live. Không khẳng định live không đổi khi chưa kiểm chứng. Xem [docker-compose.yml](#).

Câu hỏi bổ sung

11. Chưa có nhãn thì có biết model khỏe hay không?

Hiệp: “Không đủ để kết luận performance. Drift phân phối có thể quan sát được nếu có feature/vector, nhưng thiếu trusted labels thì không tính được performance đáng tin cậy. Hệ thống trả insufficient evidence và chờ, không dùng dự đoán làm nhãn.” Nhắc thêm selection bias nếu chỉ review suspicious, nên có random audit và labels riêng cho Face/Voice. Không tuyên bố random audit loại hết bias.

12. Có embedding database hay cần thêm pgvector không?

Hải: “Hiện embedding JSON nằm trong PostgreSQL, lookup template theo person ID cho verification 1:1. Không có vector DB riêng. Vector search hữu ích cho tìm danh tính 1:N, nhưng chưa phải bắt buộc với đường truy vấn hiện tại. Muốn tối ưu ở quy mô lớn cần benchmark truy vấn, storage và concurrency trước.”

13. 50.000 nhân viên có chạy được không?

Hải: “Đó là design target, chưa có load test chứng minh. Cần biết số người thi đồng thời, cadence, latency budget, kích thước capture và thời gian lưu. Không thể nhận một request demo để suy ra capacity. Cửa sổ drift 100 mẫu và overlap 80% cũng cần xem lại khi số identity lớn.”

14. Vì sao Face và Voice không dùng chung lifecycle?

Dương: “Môi trường capture, score distribution và sai số hai modality khác nhau. Mic thay đổi không buộc Face retrain. Tách state, alias, persistence và routing giúp một modality rollout trong khi modality kia giữ ổn định. Integrity decision vẫn tổng hợp theo logic sản phẩm.”

15. Làm sao biết candidate tốt hơn nếu dùng lại dữ liệu train?

Hiệp: “Threshold được chọn qua calibration/CV theo identity, giữ holdout riêng ngoài tuning. Candidate và champion được so trên cùng holdout để tránh so hai metric trên hai tập khác nhau. Sau đó mới dùng shadow/canary có labels. Dataset demo vẫn có giới hạn đại diện, nên kết quả không thay thế customer benchmark.”

16. Rollback có ngay lập tức trong mọi trường hợp không?

Dương: “Gate fail đặt challenger traffic về không và giữ champion, nhưng nhãn đến muộn được kiểm tra theo tick hàng giờ. In-flight request không bị thu hồi. Nếu DB không hoạt động thì hệ thống không thể ghi trạng thái routing bền vững ngay. Vì vậy không có bảo đảm zero-risk hoặc rollback tức thì cho mọi sự cố.”

17. MLflow alias và DB không đồng bộ khi crash thì sao?

Dương: “Promotion ghi intent PROMOTING trước khi thao tác Registry, rồi mới đổi champion trong DB. Retry/reconciliation tiếp tục intent đó; serving giữ incumbent cho tới khi DB switch commit. Không coi đây là một transaction ACID xuyên hai hệ thống. Các lỗi ngoài đường promotion vẫn cần recovery và kiểm chứng.”

18. Chứng minh test Windows và deploy thật đã chạy thế nào?

Hải: “Mở đúng run GitHub FSB, xem deployment-preflight trên Windows và JUnit không skip. Deploy thật là job deploy-demo riêng cùng artifact monitoring. Ubuntu quality xanh không đủ chứng minh test chỉ dành cho Windows đã thực thi. Lấy bằng chứng theo đúng SHA/run, không lấy số test local làm trạng thái GitHub.”

19. Có fairness hoặc Responsible AI gate chưa?

Hiệp: “DAG sinh RAI report nhưng candidate endpoint lifecycle mới chưa enforce cờ fairness legacy. Chưa có demographic labels đủ để tuyên bố fairness. Nhóm ghi đây là limitation cần xử lý trước pilot, không trình bày report generation như gate đã được thực thi.”

20. Tại sao không tự retrain encoder cho đầy đủ?

Dương: “Phạm vi hiện có là threshold calibration vì thiếu raw data và trainer được kiểm chứng cho encoder fine-tuning. Huấn luyện encoder cần consented dataset, identity splits, compute, đánh giá và serving compatibility riêng. Nhóm hoàn thiện lifecycle thật cho policy hiện có trước khi mở rộng loại model.”

21. Tính mới hoặc giá trị môn học nằm ở đâu?

Dương: “Giá trị nằm ở tích hợp service và vòng đời có bằng chứng: tenant-scoped checks, lineage snapshot, reviewed drift decisions, template versioning, guarded policy rollout và rollback. Nhóm không tuyên bố thuật toán encoder hay PSI/MMD do mình phát minh. Các pretrained model và thư viện đều có nguồn được ghi trong repo.”

22. Chi phí, ROI hay mức giảm cheating là bao nhiêu?

Đức/Hải: “Nhóm chưa có phép đo business outcome trên doanh nghiệp thật. Pilot nên đo thời gian review, tỷ lệ kết luận nhầm, latency, cost theo request và phản hồi người dùng. Báo cáo tính toán giả định phải ghi rõ giả định, không biến thành số liệu tiết kiệm đã đạt được.”

23. Làm thế nào chứng minh mọi thành viên đóng góp?

Dương: “Phân công hiện có trong CONTRIBUTING.md. Bằng chứng cần gồm phần việc thực tế, PR/review, sản phẩm bàn giao và khả năng giải thích/demo. Metadata Git sau rewrite không tự chứng minh ai thực hiện phần việc ban đầu. Nhóm cần trình bày trung thực công việc từng người đã làm và phần đang chịu trách nhiệm.”

Câu kết sau Q&A

“Nhóm xin ghi nhận các góp ý về dữ liệu thật, fairness gate và kiểm thử tải. Những phần đó là điều kiện cho pilot tiếp theo. Phạm vi đã trình diễn hôm nay là dịch vụ xác minh và cơ chế kiểm soát vòng đời policy trên môi trường demo. Nhóm xin cảm ơn.”